Chapter 2 — SELECT Basics: Reading, Filtering, Sorting and Paging

This chapter covers the single-table `SELECT` completely: choosing and computing columns, filtering with `WHERE`, operators, the rules of `NULL`, `DISTINCT`, `ORDER BY`, and paging with `LIMIT`/`OFFSET` or the standard `FETCH`. Joins, grouping and subqueries (queries nested inside another query) build on these pieces in chapters 4–6.

Sections: 2.1 `SELECT` and aliases · 2.2 `WHERE` and logical operators · 2.3 Operators · 2.4 `BETWEEN`, `IN`, `LIKE`/`ILIKE`, regex · 2.5 `NULL` and three-valued logic · 2.6 `DISTINCT` / `DISTINCT ON` · 2.7 `ORDER BY` · 2.8 `LIMIT`/`OFFSET`/`FETCH` · 2.9 Logical processing order.

Setup. Examples use the `company` database from chapter 1, section 1.11. Files are in `code/`; each starts with `\pset null '(null)'` so missing values print as `(null)`. To follow along: `bash tools_pg.sh psql -d company`, then `\pset null '(null)'`.

2.1 SELECT: choosing what comes back

What it is

`SELECT` reads rows and returns a new, temporary table called the **result set**: it chooses which columns you see (and, with `WHERE`, which rows).

```
SELECT column1, column2, ...     -- what to show
FROM   table_name;               -- where to read it
```

How it works

`FROM` names the table; the `SELECT` list says which columns (or computed values) to return, in order; `*` means "every column, in table order".

```
-- code/01_select_columns.sql
SELECT * FROM products;
```

Output:

```
 product_id |           name             |  category   | unit_price | discontinued
------------+----------------------------+-------------+------------+--------------
          1 | Laptop Pro 14              | Electronics |    1299.00 | f
          2 | Wireless Mouse             | Electronics |      24.99 | f
          ...
          8 | Desk Lamp                  | Furniture   |      35.00 | t
          9 | Gift Card                  | (null)      |      50.00 | f
(9 rows)
```

(`f`/`t` are psql's display of a **boolean**, a true/false value.) To choose columns, list them: `SELECT first_name, last_name, job_title FROM employees;` returns those three columns for all 15 rows.

Expressions and column aliases

The `SELECT` list can hold any **expression** — arithmetic, function calls (a *function* is a named built-in operation such as `round(x, 2)`), constants. An **alias** (`AS new_name`) names the output column; double-quote it to keep spaces or capitals.

```sql
SELECT name,
       unit_price,
       unit_price * 1.18        AS price_with_gst,
       'per unit'               AS note,
       unit_price * 1.18 AS "Price incl. GST"
FROM products
WHERE product_id <= 3;
```

Output:

```
      name      | unit_price | price_with_gst |   note   | Price incl. GST
----------------+------------+----------------+----------+-----------------
 Laptop Pro 14  |    1299.00 |      1532.8200 | per unit |       1532.8200
 Wireless Mouse |      24.99 |        29.4882 | per unit |         29.4882
 USB-C Hub      |      49.50 |        58.4100 | per unit |         58.4100
(3 rows)
```

(GST is India's 18 % sales tax.) The table is not changed — computed columns exist only in the result. (Rounding: `round(x, 2)`, chapter 3.)

Table aliases give a table a short name, essential once queries involve several tables. `AS` is optional for both kinds, but writing it for column aliases avoids a nasty bug (*Common mistakes*).

```sql
SELECT e.first_name, e.salary / 12 monthly
FROM employees AS e
WHERE e.emp_id IN (1, 2);
```

Output:

```
 first_name |      monthly
------------+--------------------
 Asha       | 20833.333333333333
 Vikram     | 15416.666666666667
(2 rows)
```

`SELECT` without `FROM` — PostgreSQL, MySQL, SQLite and SQL Server evaluate expressions directly (psql as a calculator). Oracle required `FROM dual` (a one-row dummy table) until 23ai made `FROM` optional. `now()` returns the current date-time; `::date` **casts** (converts) it to a plain date.

```sql
SELECT 2 + 3 AS five, now()::date >= DATE '2025-01-01' AS after_2025;
```

Output:

```
 five | after_2025
------+------------
    5 | t
(1 row)
```

Common mistakes

- `SELECT *` **in application code** — fetches unneeded columns (more I/O — disk input/output — and network traffic) and breaks or silently changes when a column is added. Fine for exploring in psql.
- **A missing comma turns a column into an alias.** `SELECT first_name last_name FROM employees` returns *one* column named `last_name` holding first names. Writing `AS` makes this visible.
- **Single-quoting aliases.** `'total'` is a string literal (a fixed text value); aliases use double quotes: `"Total"`.

Self-check

1. *Does* `SELECT unit_price * 2 FROM products` *change prices?* → No; it only computes a result value.
2. *How do you name an output column* `Net Price` *exactly?* → `AS "Net Price"`.

2.2 WHERE: filtering rows

What it is

`WHERE condition` keeps only rows for which the condition is **true**. It works identically in `SELECT`, `UPDATE` and `DELETE` (chapter 9), so a filter tested with `SELECT` can be reused safely in a `DELETE` (without `WHERE`, a `DELETE` removes all rows).

How it works

Per row, the condition evaluates to `TRUE`, `FALSE` or — with `NULL` — `UNKNOWN` (2.5). Only `TRUE` rows pass.

```sql
-- code/02_where.sql
SELECT name, category, unit_price
FROM products
WHERE category = 'Electronics';
```

Output:

```
           name             |  category   | unit_price
----------------------------+-------------+------------
 Laptop Pro 14              | Electronics |    1299.00
 Wireless Mouse             | Electronics |      24.99
 Noise-Cancelling Headphones | Electronics |     199.95
(3 rows)
```

Comparisons work on numbers, text and dates. Date literals are ISO-format strings `'YYYY-MM-DD'`, converted to the column's type:

```sql
SELECT first_name, hire_date FROM employees WHERE hire_date >= '2023-01-01' ORDER BY hire_date;
```

Output:

```
 first_name | hire_date
------------+------------
 Li         | 2023-02-14
 Arjun      | 2023-11-20
 Sara       | 2024-04-01
 Rohan      | 2025-06-02
(4 rows)
```

Combining conditions: AND, OR, NOT

Operator	True when	Example
`AND`	both sides are true	`dept_id = 2 AND salary > 70000`
`OR`	at least one side is true	`city = 'Delhi' OR city = 'Mumbai'`
`NOT`	the condition is false	`NOT discontinued`

Precedence: `NOT` binds tightest, then `AND`, then `OR` (like `*` before `+`), so `a OR b AND c` means `a OR (b AND c)`:

```sql
-- Intended: Engineering or Sales staff earning over 100k
SELECT first_name, dept_id, salary
FROM employees
WHERE dept_id = 1 OR dept_id = 2 AND salary > 100000
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Rahul      |       1 |  98000.00
 Meera      |       1 |  98000.00
 Daniel     |       2 | 150000.00
(5 rows)
```

Rahul and Meera earn under 100k but appear: the query says "*everyone* in dept 1, or dept-2 staff over 100k". Writing `WHERE (dept_id = 1 OR dept_id = 2) AND salary > 100000` returns only Vikram, Neha and Daniel. Rule: whenever `AND` and `OR` mix, add parentheses.

`NOT` negates the condition after it:

```sql
SELECT name FROM products WHERE NOT discontinued AND NOT unit_price < 100 ORDER BY name;
```

Output:

```
           name
----------------------------
 Ergonomic Chair
 Laptop Pro 14
 Noise-Cancelling Headphones
 Standing Desk
(4 rows)
```

Case sensitivity

In PostgreSQL `=` on text is **case-sensitive**: `'Mumbai' = 'mumbai'` is false, so `WHERE city = 'Mumbai'` finds one of our two Mumbai customers:

```
 exact_mumbai
--------------
            1
```

MySQL's default collations (text comparison and sorting rules, 2.7) are case-*insensitive*, so the same query returns 2 there — a portability trap. In PostgreSQL use `lower(city) = 'mumbai'` or `ILIKE` (2.4).

Common mistakes

- Mixing `AND` and `OR` without parentheses.
- Comparing with `= NULL` — never true; use `IS NULL` (2.5).
- `"Mumbai"` (double quotes) for a string: PostgreSQL reads a column name and errors *column "Mumbai" does not exist*.

Self-check

1. *How is `a AND b OR c` evaluated?* → `(a AND b) OR c`.

2.3 Operators

What it is

An **operator** is a symbol or keyword combining one or more values (its **operands**) into a new value: `+`, `=`, `AND`, `||`.

Arithmetic operators

Operator	Meaning	Example	Result
`+`	add	`5 + 3`	8
`-`	subtract	`10 - 4`	6
`*`	multiply	`3 * 7`	21
`/`	divide	`20 / 5`	4
`%`	remainder (modulo)	`17 % 3`	2
`^`	power (PostgreSQL)	`2 ^ 10`	1024
`|/`, `@`	square root, absolute value (PostgreSQL)	`|/ 25`, `@ -5`	5, 5

```sql
-- code/08_operators.sql (part)
SELECT 7 / 2 AS int_div, 7 / 2.0 AS num_div, 7::numeric / 2 AS cast_div, -7 / 2 AS neg_int_div;
SELECT 2 + 3 * 4 AS no_parens, (2 + 3) * 4 AS parens;
```

Output:

```
 int_div |      num_div       |      cast_div      | neg_int_div
---------+--------------------+--------------------+-------------
       3 | 3.5000000000000000 | 3.5000000000000000 |          -3
(1 row)

 no_parens | parens
-----------+--------
        14 |     20
(1 row)
```

Integer division truncates toward zero (drops the fraction) in PostgreSQL, SQL Server and SQLite: `7 / 2` = `3`, `-7 / 2` = `-3`. A decimal operand (`2.0`) or a cast to `numeric` (PostgreSQL's exact-decimal type) keeps the fraction. MySQL differs: `/` always returns a decimal (`7 / 2` = `3.5000`) and `DIV` does integer division — a classic "my average is wrong" bug when porting.

Operators work row by row on columns — the value of each order line:

```sql
SELECT order_id, product_id, quantity, unit_price, discount,
       quantity * unit_price * (1 - discount) AS line_total
FROM order_items WHERE order_id IN (102, 104) ORDER BY order_id, product_id;
```

Output:

```
 order_id | product_id | quantity | unit_price | discount | line_total
----------+------------+----------+------------+----------+------------
      102 |          7 |        2 |     199.95 |     0.10 |   359.9100
      104 |          5 |        1 |     479.00 |     0.00 |   479.0000
      104 |          6 |        1 |     349.99 |     0.05 |   332.4905
(3 rows)
```

Comparison operators

Operator	Meaning	Example
`=`	equal	`unit_price = 49.50`
`<>` or `!=`	not equal (`<>` is standard)	`status <> 'cancelled'`
`<`, `>`, `<=`, `>=`	ordering	`salary >= 100000`
`BETWEEN a AND b`	inclusive range	2.4
`IN (…)`	membership in a list	2.4
`LIKE`, `ILIKE`, `~`	pattern match	2.4
`IS [NOT] NULL`, `IS [NOT] DISTINCT FROM`	NULL-aware tests	2.5

Logical operators

`AND`, `OR`, `NOT` — precedence in 2.2, NULL behaviour in 2.5.

String concatenation

`||` joins strings in PostgreSQL, SQLite and Oracle: `SELECT first_name || ' ' || last_name AS full_name FROM employees` → `Asha Rao`, `Vikram Menon`, …. In MySQL `||` is logical OR by default (use `CONCAT()`); SQL Server uses `+` or `CONCAT()`. Chapter 3 covers `concat()` and NULL handling.

Compound assignment operators

T-SQL (SQL Server) has shorthand `SET @x += 3`, also `-=`, `*=`, `/=`, `%=`, for **procedural variables** (named values in step-by-step code such as a stored procedure) and in `UPDATE … SET col += 1`. They are **not** standard SQL; PostgreSQL writes `SET col = col + 1`, or `x := x + 3` inside a PL/pgSQL function (PostgreSQL's built-in procedural language; chapter 14).

Dialect	Add-and-assign
PostgreSQL	`UPDATE t SET n = n + 3`; PL/pgSQL `x := x + 3`
SQL Server	`SET @x += 3` ; `UPDATE t SET n += 3`
MySQL	`SET @x = @x + 3` (no `+=`)

Operator precedence (highest first, simplified)

Level	Operators
1	`::` (cast)
2	unary `-` / `+`
3	`^`
4	`*`, `/`, `%`
5	binary `+`, `-`
6	other operators such as `||`
7	`BETWEEN`, `IN`, `LIKE`, `ILIKE`
8	`<`, `>`, `=`, `<=`, `>=`, `<>`
9	`IS NULL`, `IS DISTINCT FROM` …
10	`NOT`
11	`AND`
12	`OR`

(*Unary* operators take one operand, like `-5`; *binary* take two.) Because unary minus binds tighter than `^`, `SELECT -2 ^ 2` returns `4` in PostgreSQL, not `-4`. When in doubt, parenthesise.

Self-check

1. *`SELECT 9 / 4` in PostgreSQL?* → `2`; `9 / 4.0` → `2.25`.
2. *Why might `first_name || ' ' || last_name` misbehave in MySQL?* → `||` is logical OR there by default.

2.4 Ranges, lists and patterns: BETWEEN, IN, LIKE/ILIKE, regex

BETWEEN — inclusive ranges

`x BETWEEN a AND b` means `x >= a AND x <= b`. **Both ends are included.**

```sql
-- code/03_between_in_like.sql
SELECT name, unit_price FROM products WHERE unit_price BETWEEN 49.50 AND 199.95 ORDER BY unit_price;
```

Output:

```
           name             | unit_price
----------------------------+------------
 USB-C Hub                  |      49.50
 Gift Card                  |      50.00
 Mechanical Keyboard        |      89.00
 Noise-Cancelling Headphones |     199.95
(4 rows)
```

`NOT BETWEEN` is the complement. `a` must be the smaller value — `BETWEEN 200 AND 50` matches nothing (PostgreSQL's `BETWEEN SYMMETRIC` accepts either order).

The timestamp trap. `placed_at` is a timestamp (date *and* time). `'2025-01-31'` means midnight at the *start* of 31 January, so an order at 23:55 that day is excluded:

```sql
SELECT order_id, placed_at FROM orders
WHERE placed_at BETWEEN '2025-01-01' AND '2025-01-31'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
(2 rows)
```

Order 103 (31 Jan, 23:55) is missing. The robust pattern is **half-open**: `>=` the start and `<` the next period's start.

```
SELECT order_id, placed_at FROM orders
WHERE placed_at >= '2025-01-01' AND placed_at < '2025-02-01'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
      103 | 2025-01-31 23:55:00
(3 rows)
```

IN — membership in a list

`x IN (a, b, c)` is shorthand for `x = a OR x = b OR x = c`; `NOT IN` negates it.

```
SELECT order_id, status FROM orders WHERE status IN ('pending', 'cancelled') ORDER BY order_id;
```

Output:

```
 order_id |  status
----------+-----------
      105 | cancelled
      111 | cancelled
      113 | pending
      114 | pending
(4 rows)
```

`WHERE job_title NOT IN ('Engineer', 'Sales Rep', 'Intern')` likewise returns the 9 other employees. `IN` also takes a subquery (`WHERE customer_id IN (SELECT …)`, chapter 6). **Warning:** `NOT IN` with a `NULL` in the list returns no rows (2.5).

LIKE and ILIKE — simple patterns

Wildcard	Matches	Example	Matches…
`%`	any sequence of zero or more characters	`'%Desk%'`	"Standing Desk", "Desk Lamp"
`_`	exactly one character	`'_a%'`	"Daniel", "Rahul", "Sara" (second letter *a*)

```
SELECT name FROM products WHERE name LIKE '%Desk%' ORDER BY name;
```

Output:

```
     name
--------------
 Desk Lamp
 Standing Desk
(2 rows)
```

`LIKE` is **case-sensitive** in PostgreSQL; `ILIKE` (PostgreSQL-only) ignores case:

```
SELECT email FROM employees WHERE email LIKE 'neha%';
SELECT email FROM employees WHERE email ILIKE 'neha%';
```

Output:

```
 email
-------
(0 rows)

          email
---------------------------
 Neha.Gupta@Company.example
(1 row)
```

Portable alternative: `lower(email) LIKE 'neha%'`.

Matching a literal `%` or `_` — use an **escape character** ("treat the next character as plain text"). PostgreSQL's default is backslash; `ESCAPE` chooses another:

```
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10\%%' ORDER BY label;
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10!_%' ESCAPE '!' ORDER BY label;
```

Output:

```
  label
---------
 10% off
(1 row)

 label
--------
 10_off
(1 row)
```

(`VALUES (…), (…) AS t(label)` builds a tiny inline table — handy for experiments.)

Regular expressions (PostgreSQL)

A **regular expression** (regex) is a mini-language for text patterns — `^` "start of text", `[A-Z]` "one capital letter", `+` "one or more of the previous"; POSIX is the Unix standard fixing this syntax. Operators: `~` (case-sensitive match), `~*` (case-insensitive), `!~` and `!~*` (no match). Names made of a capitalised word, a space, then another capital:

```
SELECT name FROM products WHERE name ~ '^[A-Z][a-z]+ [A-Z]' ORDER BY name;
```

Output:

```
        name
--------------------
 Desk Lamp
 Ergonomic Chair
 ...
 Wireless Mouse
(7 rows)
```

`USB-C Hub` fails (`S` isn't lower case) and `Noise-Cancelling…` fails (hyphen, not space). The standard's `SIMILAR TO` exists but is rarely used.

Performance note

`LIKE 'abc%'` (fixed prefix) can use a B-tree **index** (a sorted lookup structure beside the table that lets the database jump to matching rows; **B-tree** is the default, ordered kind) — in PostgreSQL the index needs the `text_pattern_ops` operator class (tells the index how to compare values) unless the database

uses the `C` collation (byte-by-byte comparison). `LIKE '%abc'` (leading wildcard) cannot, and scans every row. Chapter 12 shows trigram (`pg_trgm`) indexes for "contains" searches (a *trigram* is every 3-character slice of the text).

Dialect notes

Feature	PostgreSQL	MySQL	SQLite	SQL Server
`LIKE` case-sensitivity	sensitive	depends on collation (default insensitive)	insensitive for ASCII (plain English letters) by default	depends on collation (default insensitive)
Case-insensitive LIKE	`ILIKE`	`LIKE` (default)	`LIKE`	`LIKE` (default)
Regex	`~`, `~*`	`REGEXP` / `RLIKE`	`REGEXP` only when a `regexp()` function is loaded (the `sqlite3` shell includes one)	none built in (SQL Server 2025 adds `REGEXP_LIKE`)

Common mistakes

- `BETWEEN` on timestamps with date-only upper bounds (use half-open ranges).
- Forgetting `_` is a wildcard: `LIKE 'user_%'` also matches `userA…`.

Self-check

1. *Pattern for "exactly 4 characters ending in `n`"?* → `LIKE '___n'`.
2. *Names containing `%`?* → `LIKE '%\%%'` (or `ESCAPE`).

2.5 NULL and three-valued logic

What it is

`NULL` is a **marker for a missing value** — unknown ("we don't know Meera's email") or not applicable ("interns earn no commission"). It is not zero, not an empty string, and not a value, so it cannot *equal* anything — not even another `NULL`.

Why it matters

Real data is full of gaps, and `NULL` changes every comparison, filter and calculation. Most "missing rows" bugs come from it.

How it works — three-valued logic

A comparison involving `NULL` yields the third truth value **UNKNOWN** (displayed as `NULL`). Arithmetic and concatenation with `NULL` yield `NULL`.

```sql
-- code/04_nulls.sql
SELECT NULL = NULL AS null_eq_null, NULL <> 1 AS null_ne_1, NULL IS NULL AS is_null,
       1 + NULL AS plus, 'a' || NULL AS concat;
```

Output:

```
 null_eq_null | null_ne_1 | is_null |  plus  | concat
--------------+-----------+---------+--------+--------
 (null)       | (null)    | t       | (null) | (null)
(1 row)
```

Truth tables, generated by the database:

```sql
SELECT a, b, a AND b AS "a AND b", a OR b AS "a OR b", NOT a AS "NOT a"
FROM (VALUES (true), (false), (NULL::boolean)) AS x(a)
CROSS JOIN (VALUES (true), (false), (NULL::boolean)) AS y(b);
```

Output:

```
   a    |   b    | a AND b | a OR b | NOT a
--------+--------+---------+--------+--------
 t      | t      | t       | t      | f
 t      | f      | f       | t      | f
 t      | (null) | (null)  | t      | f
 f      | t      | f       | t      | t
 f      | f      | f       | f      | t
 f      | (null) | f       | (null) | t
 (null) | t      | (null)  | t      | (null)
 (null) | f      | f       | (null) | (null)
 (null) | (null) | (null)  | (null) | (null)
(9 rows)
```

Read UNKNOWN as "could be either": `FALSE AND UNKNOWN` is `FALSE`, `TRUE OR UNKNOWN` is `TRUE`, `NOT UNKNOWN` is UNKNOWN, anything else with UNKNOWN stays UNKNOWN. `WHERE` **keeps only TRUE** — FALSE and UNKNOWN rows are dropped.

Testing for NULL: IS NULL / IS NOT NULL

```sql
SELECT count(*) AS eq_null  FROM employees WHERE email = NULL;
SELECT count(*) AS is_null  FROM employees WHERE email IS NULL;
SELECT first_name, last_name FROM employees WHERE email IS NULL ORDER BY emp_id;
```

Output:

```
 eq_null
---------
       0
(1 row)

 is_null
---------
       2
(1 row)

 first_name | last_name
------------+-----------
 Meera      | Iyer
 Ananya     | Das
(2 rows)
```

`= NULL` is always UNKNOWN, so it never matches. (SQL Server has a legacy `ANSI_NULLS OFF` mode where `= NULL` works; don't rely on it.)

The "missing rows" trap

Sales reps have commission `0.10`, `0.10` and `NULL`. `= 0.10` and `<> 0.10` should cover everyone — but Sara (NULL) is in neither:

```sql
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct =  0.10 ORDER BY
emp_id;
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct <> 0.10 ORDER BY
emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Priya      |           0.10
 Arjun      |           0.10
(2 rows)

 first_name | commission_pct
------------+----------------
(0 rows)
```

`IS DISTINCT FROM` is the NULL-safe "not equal": two NULLs are equal, NULL vs a value differ, and the result is always TRUE or FALSE.

```
SELECT first_name, commission_pct FROM employees
WHERE job_title = 'Sales Rep' AND commission_pct IS DISTINCT FROM 0.10 ORDER BY emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Sara       |         (null)
(1 row)
```

(MySQL's NULL-safe equality is `<=>`; SQL Server supports `IS [NOT] DISTINCT FROM` from 2022.)

The NOT IN trap

`x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`; the last part is always UNKNOWN, so the condition is never TRUE:

```
SELECT count(*) AS not_in_with_null FROM departments WHERE dept_id NOT IN (1, 2, NULL);
SELECT count(*) AS not_in_no_null   FROM departments WHERE dept_id NOT IN (1, 2);
```

Output:

```
 not_in_with_null
------------------
                0
(1 row)

 not_in_no_null
----------------
              4
(1 row)
```

It bites hardest with subqueries: `WHERE id NOT IN (SELECT manager_id FROM employees)` returns nothing because the CEO's `manager_id` is NULL. Use `NOT EXISTS` (chapter 6).

NULL is not zero or ''

```
SELECT count(*) AS rows, count(commission_pct) AS non_null_commissions FROM employees;
SELECT '' IS NULL AS empty_string_is_null;
```

Output:

```
 rows | non_null_commissions
------+----------------------
   15 |                    3
(1 row)

 empty_string_is_null
----------------------
 f
(1 row)
```

`count(*)` counts rows; `count(column)` counts non-NULL values (chapter 4). An empty string is a real value — except in **Oracle**, which stores `''` as NULL.

Allowing or forbidding NULL in a column

Columns accept NULL **unless** declared `NOT NULL`; writing `NULL` in a definition just states the default.

```
CREATE TABLE students (
    id    integer PRIMARY KEY,
    email text NULL,          -- explicit "NULL allowed" (the default anyway)
    name  text NOT NULL       -- a value is required
);
INSERT INTO students VALUES (1, NULL, 'Ravi');
INSERT INTO students VALUES (2, 'kim@school.example', NULL);
SELECT * FROM students WHERE email IS NULL;
```

Output:

```
CREATE TABLE
INSERT 0 1
ERROR:  null value in column "name" of relation "students" violates not-null constraint
DETAIL:  Failing row contains (2, kim@school.example, null).
 id | email  | name
----+--------+------
  1 | (null) | Ravi
(1 row)
```

Replacing NULLs in output (`COALESCE`, `NULLIF`): chapter 3; NULLs in sorting 2.7, in `DISTINCT` 2.6, in aggregates chapter 4, in joins chapter 5.

In the interview

30-second answer: "NULL means unknown, so comparisons with it give UNKNOWN, and WHERE only keeps TRUE rows. Use IS NULL / IS NOT NULL, or IS DISTINCT FROM for NULL-safe comparison. NOT IN with a NULL in the list or subquery returns nothing, so prefer NOT EXISTS. Aggregates like COUNT(col) and SUM skip NULLs."

- *What does `NULL = NULL` return?* → NULL (UNKNOWN), not TRUE.
- *Why can `WHERE x <> 5` miss rows?* → Rows where `x` is NULL give UNKNOWN and are filtered out.
- *Is `''` NULL?* → Not in PostgreSQL, MySQL, SQL Server or SQLite; yes in Oracle.

Self-check

1. *`TRUE OR NULL`? `FALSE AND NULL`? `NOT NULL` (boolean)?* → TRUE, FALSE, NULL.
2. *Rows returned by `WHERE email = NULL`?* → Zero, always.

2.6 DISTINCT: removing duplicate rows

What it is

`SELECT DISTINCT` removes duplicate rows from the **result** (the table is untouched) — "which categories exist?", "which cities do customers live in?".

How it works

```
-- code/05_distinct.sql
SELECT DISTINCT category FROM products ORDER BY category;
```

Output (the 9 rows collapse to 4):

```
 category
------------
 Accessories
 Electronics
 Furniture
 (null)
(4 rows)
```

Two rules:

1. **NULLs count as duplicates of each other** for `DISTINCT` (unlike `=`) — NULL categories collapse into one row.
2. `DISTINCT` **applies to the whole selected row**, not the first column — with two columns you get distinct *pairs*:

```
SELECT DISTINCT country, city FROM customers ORDER BY country, city;
```

Output:

```
 country |   city
---------+-----------
 India   | Bengaluru
 India   | Delhi
 India   | Mumbai
 India   | mumbai
 India   | (null)
 UK      | London
 USA     | New York
(7 rows)
```

`Mumbai` and `mumbai` both survive — `DISTINCT` removes only **exact** duplicates. Clean dirty data first (`trim` strips leading/trailing spaces; `lower` lower-cases). Anita Shah was entered twice with differently formatted emails:

```
SELECT DISTINCT first_name, last_name, email FROM customers WHERE last_name = 'Shah';
SELECT DISTINCT first_name, last_name, lower(trim(email)) AS email FROM customers WHERE last_name = 'Shah';
```

Output:

```
 first_name | last_name |           email
------------+-----------+---------------------------
 Anita      | Shah      |   Anita.Shah@Mail.example
 Anita      | Shah      | anita.shah@mail.example
(2 rows)

 first_name | last_name |          email
------------+-----------+------------------------
 Anita      | Shah      | anita.shah@mail.example
(1 row)
```

Counting distinct values: `count(DISTINCT expr)`, which ignores NULLs (aggregates — functions such as `count` and `sum` that combine many rows into one value — are chapter 4):

```
SELECT count(*) AS rows, count(city) AS with_city, count(DISTINCT city) AS distinct_cities,
       count(DISTINCT lower(city)) AS distinct_cities_ci
FROM customers;
```

Output:

```
 rows | with_city | distinct_cities | distinct_cities_ci
------+-----------+-----------------+--------------------
   10 |         9 |               6 |                  5
(1 row)
```

DISTINCT ON (PostgreSQL)

`DISTINCT ON (expr)` keeps the **first row of each group** of equal `expr` values, "first" being decided by `ORDER BY` (which must start with the same expressions). Top earner per department:

```
SELECT DISTINCT ON (dept_id) dept_id, first_name, salary
FROM employees
WHERE dept_id IS NOT NULL
ORDER BY dept_id, salary DESC, emp_id;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       2 | Daniel     | 150000.00
       3 | Lucas      | 120000.00
       4 | Kiran      | 135000.00
       5 | Li         |  64000.00
(5 rows)
```

Other databases use window functions (which look at related rows without merging them, e.g. `ROW_NUMBER()` — chapter 8).

Trade-offs and when not to use it

- `DISTINCT` sorts or hashes the whole result (hashing turns each row into a number used to bucket equal rows) — expensive on millions of rows.
- **Don't use `DISTINCT` to hide a bad join.** Unexpected duplicates usually mean a wrong join condition (*fan-out*, chapter 5); `DISTINCT` masks the symptom and can hide real duplicates.
- For counts per value use `GROUP BY` (chapter 4), not `DISTINCT`.

Self-check

1. *Does `SELECT DISTINCT a, b` make `a` unique?* → No; each `(a, b)` pair.
2. *Why did `count(DISTINCT city)` give 6 but the case-insensitive version 5?* → `Mumbai` and `mumbai` match only after `lower()`.

2.7 ORDER BY: sorting results

What it is

Without `ORDER BY`, rows come back in **no guaranteed order** — whatever is fastest at that moment. `ORDER BY` is the only way to get a defined order. `ASC` (smallest first) is the default; `DESC` is descending.

```
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC];
```

How it works

```
-- code/06_order_by.sql
SELECT first_name, salary FROM employees ORDER BY salary DESC;
```

Output (abridged):

```
 first_name |  salary
------------+----------
 Asha       | 250000.00
 Vikram     | 185000.00
 ...
 Meera      |  98000.00
 Rahul      |  98000.00
 ...
 Rohan      |  30000.00
(15 rows)
```

Meera and Rahul tie; their relative order is **not guaranteed**. Add a unique **tie-breaker** column whenever order matters (especially for paging) so the order is **deterministic** — the same every run.

Several sort keys — the second key only matters within ties of the first, each with its own direction:

```
SELECT dept_id, first_name, salary FROM employees
ORDER BY dept_id, salary DESC, first_name;
```

Output (abridged):

```
 dept_id | first_name |  salary
---------+------------+----------
       1 | Vikram     | 185000.00
       1 | Neha       | 142000.00
       1 | Meera      |  98000.00
       1 | Rahul      |  98000.00
       2 | Daniel     | 150000.00
 ...
       5 | Li         |  64000.00
  (null) | Asha       | 250000.00
  (null) | Rohan      |  30000.00
(15 rows)
```

Where do NULLs go?

PostgreSQL treats NULL as **larger than any value**: **last in ASC**, **first in DESC**. Override with `NULLS FIRST` / `NULLS LAST`:

```
SELECT name, location FROM departments ORDER BY location NULLS FIRST;
```

Output:

```
    name     | location
-------------+-----------
 Support     | (null)
 Engineering | Bengaluru
 Sales       | Mumbai
 Marketing   | Mumbai
 Finance     | Pune
 Research    | Pune
(6 rows)
```

(Plain `ORDER BY location` puts Support last; `ORDER BY location DESC NULLS LAST` gives Pune first and Support last.)

Database	NULLs in ASC	`NULLS FIRST/LAST` supported?
PostgreSQL, Oracle	last	yes
MySQL, SQL Server, SQLite	first	SQLite yes (3.30+); MySQL/SQL Server no — use `ORDER BY col IS NULL, col` (MySQL) or a `CASE` expression

Sorting by alias, expression or position

```
SELECT name, unit_price * 1.18 AS gross FROM products ORDER BY gross DESC LIMIT 3;
SELECT first_name, last_name FROM employees ORDER BY length(last_name) DESC, last_name LIMIT 3;
SELECT first_name, hire_date FROM employees ORDER BY 2 LIMIT 3;
```

Output:

```
     name       |   gross
----------------+----------
 Laptop Pro 14  | 1532.8200
 Standing Desk  |  588.8200
 Ergonomic Chair |  412.9882
(3 rows)

 first_name | last_name
------------+-----------
 Daniel     | Brooks
 Omar       | Farouk
 Lucas      | Martin
(3 rows)

 first_name | hire_date
------------+------------
 Asha       | 2015-01-05
 Kiran      | 2016-12-01
 Daniel     | 2017-09-25
(3 rows)
```

`ORDER BY 2` means "the second select-list column" — quick in psql, fragile in code (reorder the list and the sort silently changes).

Custom order with CASE

Sort statuses by workflow, not alphabetically (`CASE` is chapter 3):

```
SELECT order_id, status FROM orders
ORDER BY CASE status WHEN 'pending' THEN 1 WHEN 'shipped' THEN 2
                    WHEN 'delivered' THEN 3 ELSE 4 END, order_id;
```

Output (abridged):

```
 order_id |  status
----------+-----------
      113 | pending
      114 | pending
      107 | shipped
      110 | shipped
      112 | shipped
      101 | delivered
      ...
      109 | delivered
      105 | cancelled
      111 | cancelled
(14 rows)
```

Text order depends on collation

A **collation** is the rule set for comparing and sorting text in a language. The book's server uses the `C` collation (byte order), so all capitals sort before all lower-case letters:

```
SELECT city FROM customers WHERE city IS NOT NULL ORDER BY city;
```

Output:

```
   city
-----------
 Bengaluru
 Delhi
 Delhi
 London
 London
 Mumbai
 New York
 New York
 mumbai
(9 rows)
```

`ORDER BY lower(city), city` puts `mumbai` next to `Mumbai`; so would a database created with a language collation such as `en_US.UTF-8`, or per query `ORDER BY city COLLATE "en-x-icu"` (needs ICU — International Components for Unicode, a library of language-aware text rules).

Common mistakes

- Relying on insertion order or "it always came back sorted".
- No unique tie-breaker, then paging (2.8) shows some rows twice and others never.
- Numbers stored as text sort alphabetically: `'10' < '9'`.

Self-check

1. *Where does PostgreSQL put NULLs with `ORDER BY x DESC`?* → First.
2. *Why add `emp_id` to `ORDER BY salary DESC`?* → Unique tie-breaker for a deterministic order.

2.8 LIMIT, OFFSET and FETCH: returning a page

What it is

`LIMIT n` returns at most `n` rows; `OFFSET m` skips the first `m`. Together they return "page *k*". The SQL-standard spelling is `OFFSET m ROWS FETCH FIRST n ROWS ONLY`. **Always pair them with `ORDER BY`** — "the first 3 rows" means nothing without an order.

How it works

```sql
-- code/07_limit_offset.sql
-- Top 3 earners
SELECT first_name, salary FROM employees ORDER BY salary DESC LIMIT 3;

-- Page 2 with 5 rows per page (skip 5)
SELECT emp_id, first_name FROM employees ORDER BY emp_id LIMIT 5 OFFSET 5;

-- Standard SQL spelling of the same thing
SELECT emp_id, first_name FROM employees ORDER BY emp_id
OFFSET 5 ROWS FETCH FIRST 5 ROWS ONLY;
```

Output:

```
 first_name |  salary
------------+----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
(3 rows)

 emp_id | first_name
--------+------------
      6 | Daniel
      7 | Priya
      8 | Arjun
      9 | Sara
     10 | Lucas
(5 rows)
```

(The `FETCH` version returns the same 5 rows.) Page *p* (from 1) of size *s*: `LIMIT s OFFSET (p - 1) * s`.

Ties: FETCH … WITH TIES

`LIMIT 1` on a tie picks one row arbitrarily. `FETCH FIRST n ROWS WITH TIES` (PostgreSQL 13+, requires `ORDER BY`) also returns every row tying with the last:

```
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary LIMIT 1;
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary FETCH FIRST 1 ROWS WITH TIES;
```

Output:

```
 first_name |  salary
------------+----------
 Arjun      | 68000.00
(1 row)

 first_name |  salary
------------+----------
 Arjun      | 68000.00
 Sara       | 68000.00
(2 rows)
```

"Lowest-paid person" has two correct answers; `WITH TIES` returns both (chapter 8's ranking functions generalise this).

Why OFFSET gets slow — keyset pagination

`OFFSET 100000` still produces and discards 100,000 rows, so deep pages slow down; rows inserted or deleted between page loads shift pages, so users see duplicates or miss rows. **Keyset** ("seek") pagination remembers the last key shown and asks for rows after it:

```
-- page after the one that ended at emp_id 10
SELECT emp_id, first_name FROM employees WHERE emp_id > 10 ORDER BY emp_id LIMIT 5;
```

Output:

```
 emp_id | first_name
--------+------------
     11 | Ananya
     12 | Kiran
     13 | Omar
     14 | Li
     15 | Rohan
(5 rows)
```

With an index on the key, every page costs the same however deep. Trade-off: "next/previous" only, no jump to page 500. Chapter 12 has details.

Dialect differences

Database	First 10 rows	Skip 20, take 10
PostgreSQL	`LIMIT 10` or `FETCH FIRST 10 ROWS ONLY`	`LIMIT 10 OFFSET 20` or `OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
MySQL	`LIMIT 10`	`LIMIT 10 OFFSET 20` or `LIMIT 20, 10` (offset first!)
SQLite	`LIMIT 10`	`LIMIT 10 OFFSET 20`
SQL Server	`SELECT TOP (10) …`	`ORDER BY … OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
Oracle 12c+	`FETCH FIRST 10 ROWS ONLY`	`OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`

Common mistakes

- `LIMIT` without `ORDER BY` — rows that can change between runs.
- Reading MySQL's `LIMIT 20, 10` as "20 rows starting at 10" — it's the reverse.
- Paging on a non-unique sort key — boundary rows repeat or vanish.

In the interview

> **30-second answer:** "`ORDER BY` plus `LIMIT n OFFSET m`, or the standard `OFFSET … FETCH`. Deep OFFSET is O(offset) (work grows in proportion to the offset) and unstable under inserts, so I'd use keyset pagination — `WHERE (sort_key, id) > (:last_key, :last_id) ORDER BY sort_key, id LIMIT n` — backed by an index." (`:last_key` / `:last_id` are placeholders the application fills with the last row shown.)

- *Top 3 including ties?* → `FETCH FIRST 3 ROWS WITH TIES`, or `DENSE_RANK()` (chapter 8).
- *SQL Server equivalent of LIMIT?* → `TOP (n)` or `OFFSET … FETCH`.

Self-check

1. `LIMIT 5 OFFSET 12` *over 15 rows?* → 3 rows (13–15).
2. *Page 4, size 20?* → `LIMIT 20 OFFSET 60`.
3. *What does keyset pagination need?* → A stable, unique ordering key, ideally indexed.

2.9 How a SELECT is evaluated (logical processing order)

What it is

You *write* `SELECT … FROM … WHERE … ORDER BY … LIMIT`, but the database *evaluates* it logically in a different order, which explains many errors.



(Chapter 4 inserts `GROUP BY` and `HAVING` between `WHERE` and `SELECT`, and window functions after them.)

- `WHERE` **cannot use a column alias** from the `SELECT` list — `SELECT` hasn't happened yet.
- `ORDER BY` **can use aliases** (it runs after `SELECT`) — but in PostgreSQL only a bare alias: `ORDER BY gross` works, `ORDER BY gross + 0` fails with *column "gross" does not exist* (inside an expression, names refer to input columns).
- `LIMIT` is applied last, to the sorted result.

```sql
-- code/01_select_columns.sql (end)
SELECT name, unit_price * 1.18 AS gross FROM products WHERE gross > 500;
SELECT name, unit_price * 1.18 AS gross FROM products WHERE unit_price * 1.18 > 500 ORDER BY gross DESC;
```

Output:

```
ERROR:  column "gross" does not exist
LINE 1: ...e, unit_price * 1.18 AS gross FROM products WHERE gross > 50...
                                                             ^
     name      |   gross
---------------+-----------
 Laptop Pro 14 | 1532.8200
 Standing Desk |  588.8200
(2 rows)
```

This *logical* order defines the meaning; the optimizer (the component choosing how to run a query) may physically differ — e.g. read rows already sorted from an index — as long as the result is the same.

Putting it together

"The three most recent non-cancelled 2025 orders placed through a sales rep":

```sql
-- code/09_putting_it_together.sql
SELECT order_id, customer_id, sales_rep_id, placed_at, status
FROM orders
WHERE status <> 'cancelled'
  AND sales_rep_id IS NOT NULL
  AND placed_at >= '2025-01-01' AND placed_at < '2026-01-01'
ORDER BY placed_at DESC, order_id DESC
LIMIT 3;
```

Output:

```
 order_id | customer_id | sales_rep_id |      placed_at      | status
----------+-------------+--------------+---------------------+---------
      113 |           1 |            7 | 2025-06-02 10:30:00 | pending
      112 |           5 |            9 | 2025-05-20 21:05:00 | shipped
      110 |           7 |            8 | 2025-04-25 11:10:00 | shipped
(3 rows)
```

Chapter 2 TL;DR

- `SELECT cols FROM t` — name your columns (`*` only to explore); any expression can be a column; `AS` aliases name it; table aliases shorten references.
- `WHERE` keeps TRUE rows only. Precedence `NOT` > `AND` > `OR`: parenthesise. Text `=` is case-sensitive in PostgreSQL.
- Integer division truncates in PostgreSQL (`7/2 = 3`); `<>` is not-equal; `||` concatenates; compound `+=` is T-SQL/procedural, not standard SQL.
- `BETWEEN` is inclusive (use half-open `>=` / `<` ranges for timestamps); `IN` = ORs; `LIKE` (`%`, `_`) is case-sensitive, `ILIKE` isn't; `~` for regex.
- NULL means unknown → three-valued logic. Use `IS [NOT] NULL`, `IS [NOT] DISTINCT FROM`; never put a nullable list inside `NOT IN`.
- `DISTINCT` removes exact duplicate rows over the whole select list (NULLs grouped); clean with `lower` / `trim` first; `DISTINCT ON` keeps the first row per group.
- Only `ORDER BY` gives order; add unique tie-breakers; `NULLS FIRST/LAST`; collation affects text order.
- `LIMIT/OFFSET` or `OFFSET … FETCH`; `WITH TIES` keeps ties; keyset pagination for deep pages.
- Logical order FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT; aliases exist from SELECT onward.

Next: Chapter 3 — Functions and expressions.